

## Part II: Graph Algorithms

### Lecture 9: Dijkstra's Shortest Path Algorithm

Computer Science and Technology  
United International College

# Objective and Outline

**Objective:** Discuss a third graph problem: single-source shortest paths.

**Reference:** Section 24.3 of CLRS

- 1 The single-source shortest paths problem
- 2 Dijkstra's algorithm
- 3 Example
- 4 Correctness
- 5 Running time

# Shortest Path Problem for Weighted Graphs

- Let  $G = (V, E)$  be a **weighted** digraph, with a weight function  $w : E \rightarrow \mathbb{R}$  mapping edges to real-valued weights.
- If  $e = (u, v)$ , we write  $w(u, v)$  for  $w(e)$ .

## Definition

The **length** of a path  $p = \langle v_0, v_1, \dots, v_k \rangle$  is the **sum** of the weights on its edges

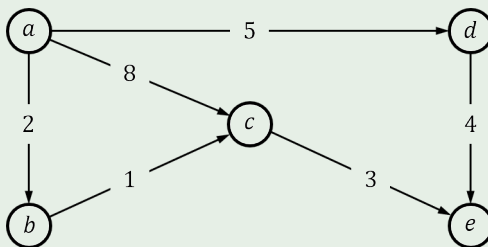
$$\text{length}(p) = \sum_{i=1}^k w(v_{i-1}, v_i)$$

# Distance

## Definition

The **distance** from  $u$  to  $v$ , denoted  $\delta(u, v)$ , is the length of the “**shortest**” path if there is a path from  $u$  to  $v$ ; and is  $\infty$  otherwise.

## Example



- $length(\langle a, b, c, e \rangle) = 6$ ; the distance from  $a$  to  $e$  is  $\delta(a, e) = 6$ .

# Single-Source Shortest-Paths Problem

## Definition (Single-source shortest-paths problem)

Given a digraphed  $G = (V, E)$  with **non-negative weighted** edges and a designated **source vertex**  $s$ ,  $s \in V$ , determine the **distance** and a **shortest path** from the source vertex to **every** vertex in the digraph.

## Question

How do you design an efficient algorithm for this problem?

# Outline

- 1 The single-source shortest paths problem
- 2 Dijkstra's algorithm
- 3 Example
- 4 Correctness
- 5 Running time

# The Rough Idea of Dijkstra's Algorithm

- Maintain  $d[v]$  and  $S$ :
  - $d[v]$  is an estimate of the length  $\delta(s, v)$  of the shortest path for each vertex  $v$ .
  - $S \subseteq V$  is a subset of vertices for which we **know** the true **distance**, that is  $d[v] = \delta(s, v)$ .
- Initially,
  - $S = \emptyset$ .
  - $d[s] = 0$  and  $d[v] = \infty$  for all others vertices  $v$ .
- **Each round**, we **select** one vertex from  $V \setminus S$  and add it to  $S$ .
- Questions to answer at each step are
  - 1 Which vertex do we select?
  - 2 How do we update the distance estimates after a vertex is added to  $S$ ?

## Question

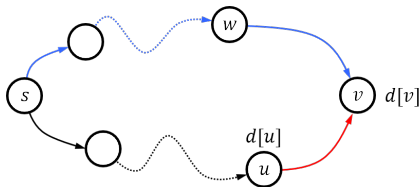
How does the algorithm **select** which vertex among the vertices of  $V \setminus S$  to process next?

**Answer:** We use a **greedy** algorithm.

- For each vertex in  $u \in V \setminus S$ , we have computed a distance estimate  $d[u]$ .
- The next processed vertex is always a vertex  $u \in V \setminus S$  for which  $d[u]$  is **minimum**.
  - In other words, we pick the unprocessed vertex which is **closest** (by our estimate) to  $s$ .

# Updating distance estimates

- $d[v]$  is the current estimated distance for  $v$ .
- If the vertex  $u$  just added to  $S$ , and  $(u, v)$  is an edge of weight  $w(u, v)$ .
- How do we update  $d[v]$ ?



- The current shortest path from  $s$  is  $\langle s, \dots, w, v \rangle$  of length  $d[v]$ .
- After picking  $u$ , new path can be  $\langle s, \dots, u, v \rangle$  of length  $d[u] + w(u, v)$ .
- If the new path is shorter than the old ( $d[u] + w(u, v) < d[v]$ ), update  $d[v] = d[u] + w(u, v)$ . This is called edge relaxation.

# The Algorithm for Relaxing an Edge

---

**Algorithm 1**  $\text{Relax}(u, v)$ 

---

```
1: if  $d[u] + w(u, v) < d[v]$  then  
2:    $d[v] = d[u] + w(u, v)$   
3:    $\text{pred}[v] = u$   
4: end if
```

---

**Remark 1:** The predecessor pointer  $\text{pred}[]$  is for determining the shortest paths.

**Remark 2:** After edge  $(u, v)$  is relaxed, we have

$$d[v] \leq d[u] + w(u, v)$$

# The Selection in Dijkstra's Algorithm

## Question

How do we perform this selection efficiently?

**Answer:** We store the vertices of  $V \setminus S$  in a **Priority queue**, where the key value of each vertex  $v$  is  $d[v]$ .

- Note: if we implement the priority queue using a heap, we can perform the operations *insert()*, *extractMin()*, and *decreaseKey()*, each in  $\mathcal{O}(\log n)$  time.

# Description of Dijkstra's Algorithm

---

## Algorithm 2 Dijkstra( $G, w, s$ )

---

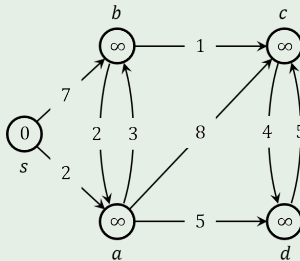
```
1: for all  $u \in V$  do    // initialize
2:    $d[u] = \infty$ 
3:    $color[u] = W$ 
4: end for
5:  $d[s] = 0$ 
6:  $pred[s] = NULL$ 
7:  $Q = \text{new PriQueue}(V)$ 
8: while  $Q$  is nonempty do    // Process all vertices
9:    $u = Q.\text{extractMin}()$     // Find new vertex
10:  for all  $v \in adj[u]$  do
11:    if  $d[u] + w(u, v) < d[v]$  then    // If estimate improves
12:       $d[v] = d[u] + w(u, v)$     // Relax
13:       $Q.\text{decreaseKey}(v, d[v])$ 
14:       $pred[v] = u$ 
15:    end if
16:  end for
17:   $color[u] = B$ 
18: end while
```

---

- 1 The single-source shortest paths problem
- 2 Dijkstra's algorithm
- 3 Example**
- 4 Correctness
- 5 Running time

# Example I

## Example



### 0 Initialization.

| $v$        | $s$        | $a$        | $b$        | $c$        | $d$        |
|------------|------------|------------|------------|------------|------------|
| $d[v]$     | 0          | $\infty$   | $\infty$   | $\infty$   | $\infty$   |
| $pred[v]$  | <i>NIL</i> | <i>NIL</i> | <i>NIL</i> | <i>NIL</i> | <i>NIL</i> |
| $color[v]$ | <i>W</i>   | <i>W</i>   | <i>W</i>   | <i>W</i>   | <i>W</i>   |

# Example II

Priority Queue:

| $v$    | $s$ | $a$      | $b$      | $c$      | $d$      |
|--------|-----|----------|----------|----------|----------|
| $d[v]$ | 0   | $\infty$ | $\infty$ | $\infty$ | $\infty$ |

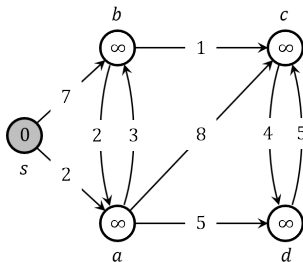
- ①  $s$  has the minimum key in the priority queue. As  $adj[s] = \{a, b\}$ , work on  $a$   $b$  and update information.

| $v$        | $s$        | $a$ | $b$ | $c$        | $d$        |
|------------|------------|-----|-----|------------|------------|
| $d[v]$     | 0          | 2   | 7   | $\infty$   | $\infty$   |
| $pred[v]$  | <i>NIL</i> | $s$ | $s$ | <i>NIL</i> | <i>NIL</i> |
| $color[v]$ | $B$        | $W$ | $W$ | $W$        | $W$        |

Priority Queue:

| $v$    | $a$ | $b$ | $c$      | $d$      |
|--------|-----|-----|----------|----------|
| $d[v]$ | 2   | 7   | $\infty$ | $\infty$ |

# Example III



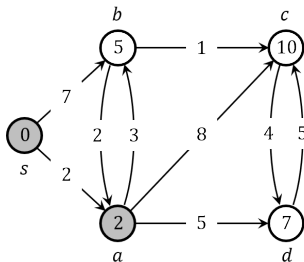
- 2  $a$  has the minimum key in the priority queue. As  $adj[a] = \{b, c, d\}$ , work on  $b$   $c$   $d$  and update information.

| $v$        | $s$        | $a$ | $b$ | $c$ | $d$ |
|------------|------------|-----|-----|-----|-----|
| $d[v]$     | 0          | 2   | 5   | 10  | 7   |
| $pred[v]$  | <i>NIL</i> | $s$ | $a$ | $a$ | $a$ |
| $color[v]$ | $B$        | $B$ | $W$ | $W$ | $W$ |

Priority Queue:

| $v$    | $b$ | $d$ | $c$ |
|--------|-----|-----|-----|
| $d[v]$ | 5   | 7   | 10  |

# Example IV



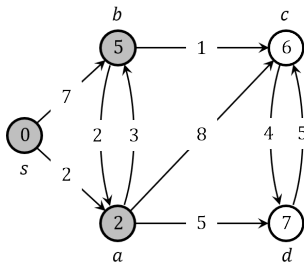
- 3  $b$  has the minimum key in the priority queue. As  $adj[b] = \{a, c\}$ , work on  $a$   $c$  and update information.

| $v$        | $s$   | $a$ | $b$ | $c$ | $d$ |
|------------|-------|-----|-----|-----|-----|
| $d[v]$     | 0     | 2   | 5   | 6   | 7   |
| $pred[v]$  | $NIL$ | $s$ | $a$ | $b$ | $a$ |
| $color[v]$ | $B$   | $B$ | $B$ | $W$ | $W$ |

Priority Queue:

| $v$    | $c$ | $d$ |
|--------|-----|-----|
| $d[v]$ | 6   | 7   |

# Example V



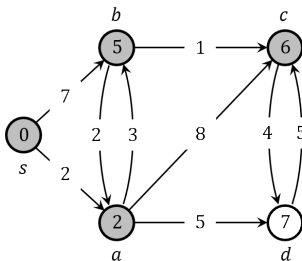
- 4  $c$  has the minimum key in the priority queue. As  $adj[c] = \{d\}$ , work on  $d$  and update information.

| $v$        | $s$   | $a$ | $b$ | $c$ | $d$ |
|------------|-------|-----|-----|-----|-----|
| $d[v]$     | 0     | 2   | 5   | 6   | 7   |
| $pred[v]$  | $NIL$ | $s$ | $a$ | $b$ | $a$ |
| $color[v]$ | $B$   | $B$ | $B$ | $B$ | $W$ |

Priority Queue:

| $v$    | $d$ |
|--------|-----|
| $d[v]$ | 7   |

# Example VI

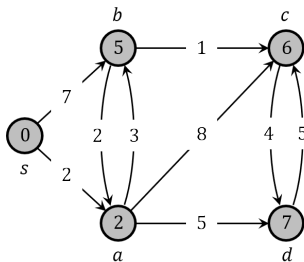


- 5  $d$  has the minimum key in the priority queue. As  $adj[d] = \{c\}$ , work on  $c$  and update information.

| $v$        | $s$        | $a$ | $b$ | $c$ | $d$ |
|------------|------------|-----|-----|-----|-----|
| $d[v]$     | 0          | 2   | 5   | 6   | 7   |
| $pred[v]$  | <i>NIL</i> | $s$ | $a$ | $b$ | $a$ |
| $color[v]$ | $B$        | $B$ | $B$ | $B$ | $B$ |

Priority Queue:  $Q = \emptyset$ . We are done.

# Example VII



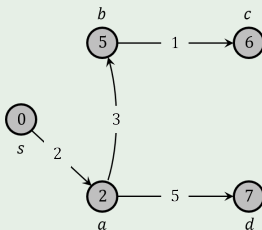
# Dijkstra's Algorithm

**Shortest Path Tree:**  $T = (V, A)$ , where

$$A = \{(pred[v], v) | v \in V \setminus \{s\}\}$$

The array  $pred[v]$  is used to build the tree.

## Example



| $v$       | $s$        | $a$ | $b$ | $c$ | $d$ |
|-----------|------------|-----|-----|-----|-----|
| $d[v]$    | 0          | 2   | 5   | 6   | 7   |
| $pred[v]$ | <i>NIL</i> | $s$ | $a$ | $b$ | $a$ |

# Outline

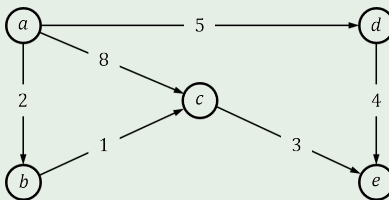
- 1 The single-source shortest paths problem
- 2 Dijkstra's algorithm
- 3 Example
- 4 Correctness**
- 5 Running time

# Key Observation

## Lemma

*Any subpath  $\langle v_1, \dots, v_i \rangle$  of a shortest path  $\langle v_1, \dots, v_i, \dots, v_j \rangle$  must also be a shortest path from  $v_1$  to  $v_i$ .*

## Example



$\langle a, b, c, e \rangle$  is a shortest path. The subpath  $\langle a, b, c \rangle$  is also a shortest path from  $a$  to  $c$ .

## Question

Why?

# Correctness of Dijkstra's Algorithm I

## Theorem

When a vertex  $u$  is added to  $S$  (i.e., dequeued from the queue),  
 $d[u] = \delta(s, u)$ .

- Dijkstra's algorithm uses **greedy** approach.
- It means that the algorithm chooses the “best” choice in each iteration. (More on greedy algorithms will be given later.)
- To prove greedy approach gives the correct (optimal) solution in general, we use induction.
- If
  - the first step is correct, and
  - if the correctness of previous steps implies the correctness of the current step;

then the correctness of the algorithm is ensured.

# Correctness of Dijkstra's Algorithm II

Proof. (by induction on the iteration of Dijkstra's algorithm)

**Base case:** When the vertex  $s$  is added to  $S$ ,  $d[s] = 0$  which is trivially same as  $\delta(s, s)$ .

**Induction hypothesis:** Assume that after iteration  $i$ ,  $d[v] = \delta(s, v)$  for all vertices  $v \in S$ .

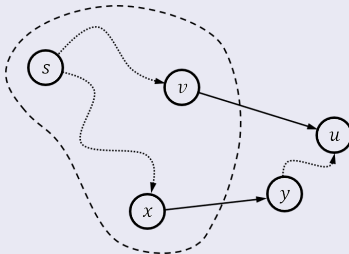
**Inductive step:** Consider the vertex  $u$  added by Dijkstra's algorithm in iteration  $i + 1$ , we prove  $d[u] = \delta(s, u)$  by contradiction.

- Suppose **negation** Dijkstra's algorithm **first** attempts to add  $u$  to  $S$  and  $d[u] \neq \delta(s, u)$ .
- By our observations about relaxation,  $d[u] > \delta(s, u)$ .

# Correctness of Dijkstra's Algorithm III

## Proof.(Con't)

- Consider the **true shortest path** from  $s$  to  $u$ , denoted as  $\langle s, \dots, u \rangle$ .
- Because  $s \in S$  and  $u \in V \setminus S$ , at some point this path must first take a jump out of  $S$ .
- Let  $(x, y)$  be the edge taken by the path, where  $x \in S$  and  $y \in V \setminus S$  ( $x = s$  is possible, but  $y \neq u$ ).
- The path becomes  $\langle s, \dots, x, y, \dots, u \rangle$ .



# Correctness of Dijkstra's Algorithm IV

## Proof. (Con't)

Since  $x \in S$  and  $y \notin S$ ,

$$d[y] = d[x] + w(x, y)$$

By induction hypothesis  $d[x] = \delta(s, x)$  if  $x \in S$ ,

$$d[y] = \delta(s, x) + w(x, y)$$

And  $y \neq u$ ,  $\delta(y, u) > 0$ ,

$$d[y] < \delta(s, x) + w(x, y) + \delta(y, u)$$

$\langle s, \dots, x \rangle$ ,  $\langle x, y \rangle$ , and  $\langle y, \dots, u \rangle$  are subpaths of  $\langle s, \dots, x, y, \dots, u \rangle$ ,

$$d[y] < \delta(s, u)$$

We have already assumed that  $\delta(s, u) < d[u]$  for prove by contradiction. Thus,  $d[y] < d[u]$ . Dijkstra's Algorithm should pick  $y$  but not  $u$ , which is a contradiction.

# Outline

- 1 The single-source shortest paths problem
- 2 Dijkstra's algorithm
- 3 Example
- 4 Correctness
- 5 Running time

# Analysis of Dijkstra's Algorithm

- The initialization uses only  $\mathcal{O}(n)$  time.
- Each vertex is processed exactly once, so *nonempty* and *extractMin()* are called exactly once, i.e.,  $n$  times in total.
- The inner loop *for all*  $v \in \text{adj}[u]$  is called once for each edge in the graph. Each call of the inner loop does  $\mathcal{O}(1)$  work plus, possibly, one *decreaseKey* operation.
- Recalling that all of the priority queue operations require  $\mathcal{O}(\log |Q|) = \mathcal{O}(\log n)$  time, we have that the algorithm uses  $n\mathcal{O}(1 + \log n) + \mathcal{O}(e) + \mathcal{O}(e \log n) = \mathcal{O}((n + e) \log n) = \mathcal{O}(e \log n)$  time.

# Minimum spanning tree vs shortest path tree

- Dijkstra's algorithm looks similar to Prim's algorithm.
- To understand the differences clearly, try them both on some example.
- Is a minimum spanning tree a shortest path tree? Vice versa?